

MODULE 0 · FOUNDATIONS

# Cluster Architecture

Control plane and worker nodes, and how they talk

---

# Every component is a loop watching one API

kubectl never "does" anything itself — it submits **desired state** to the API. Every other piece of Kubernetes is an independent control loop that watches that API and reconciles *actual* → *desired*.

Knowing which component owns which job is how you diagnose *any* cluster problem — each one fails with a distinct, recognisable signature.

## BY THE END YOU CAN

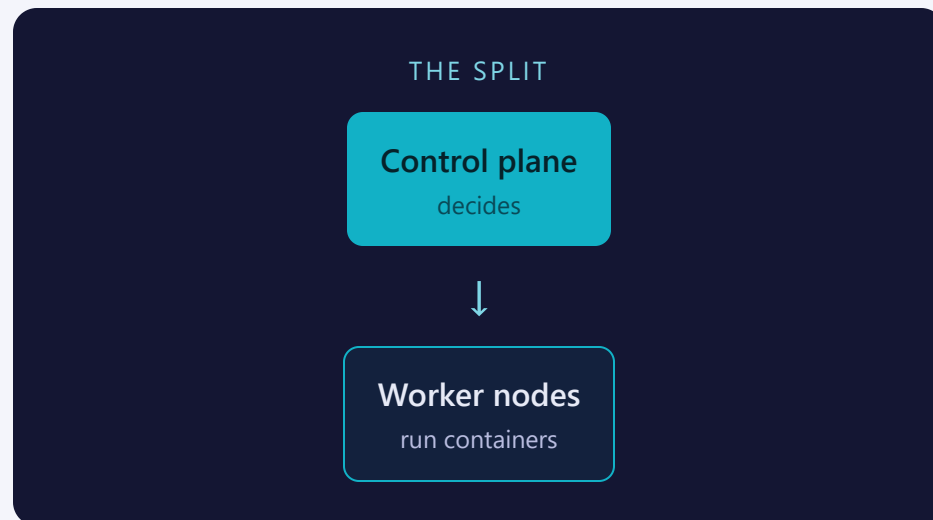
- Name the control-plane & worker components
- Explain what only the apiserver can touch
- Trace a Pod create end-to-end
- Match a symptom to the component that owns it

## CONCEPT

# Two halves: control plane and worker nodes

The **control plane** is the brain: it holds cluster state and decides what should run where. **Worker nodes** are the muscle: they run the actual containers.

One rule ties the whole system together: **everything talks through the apiserver, and only the apiserver touches etcd**. Components never call each other directly.



# Four components, one job each

| COMPONENT               | ROLE                                                                                   | IF IT'S DOWN                                          |
|-------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------|
| kube-apiserver          | Front door — REST API, authN/authZ, admission, validation; the <b>only</b> etcd client | kubectl stops working; running Pods keep serving      |
| etcd                    | Consistent key-value store holding all cluster state                                   | Cluster state is gone — no rebuild without a backup   |
| kube-scheduler          | Watches for unscheduled Pods, assigns each one a node                                  | New Pods stay Pending ; existing Pods unaffected      |
| kube-controller-manager | Runs the control loops — node, ReplicaSet, Deployment, endpoints, ServiceAccount...    | No self-healing: no replacing failed Pods, no scaling |

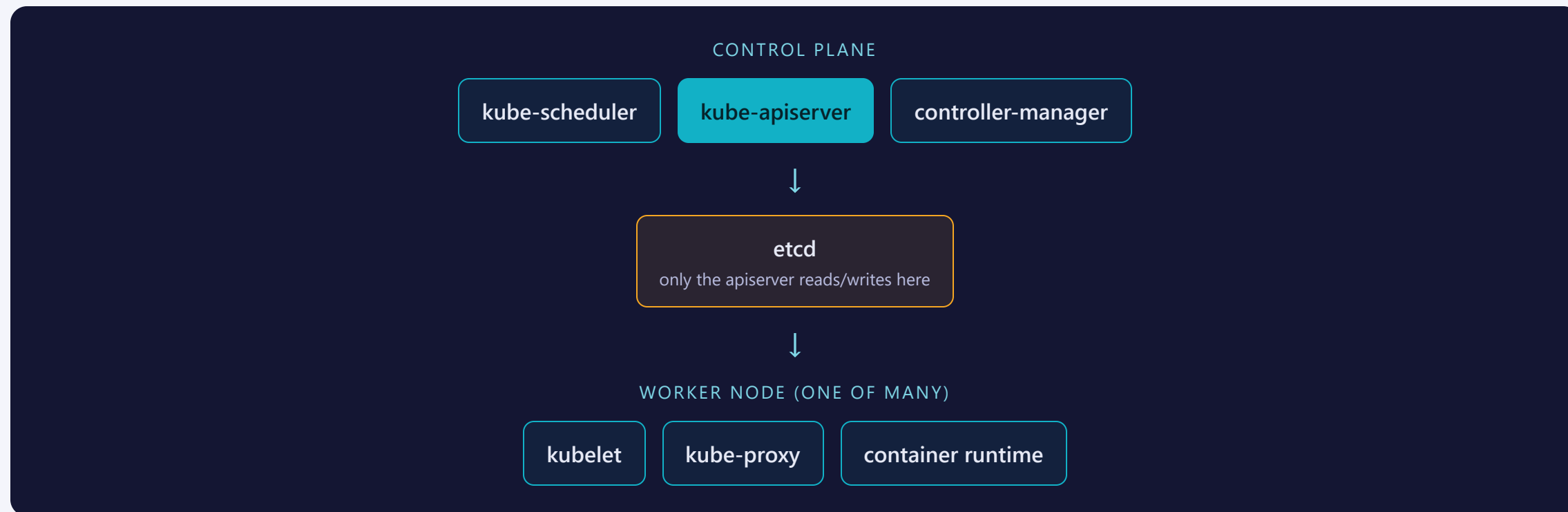
# Three components per node

| COMPONENT         | ROLE                                                                                | IF IT'S DOWN                                                                              |
|-------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| kubelet           | Node agent — drives the runtime via the CRI, runs probes, reports node & Pod status | Node goes <code>NotReady</code> ; after a grace period its Pods are rescheduled elsewhere |
| kube-proxy        | Implements Service networking on the node (iptables / IPVS / eBPF)                  | Service traffic to Pods on that node stops routing correctly                              |
| Container runtime | Pulls images and runs containers (containerd, CRI-O, ...) via the CRI               | kubelet can't start or stop containers on that node                                       |

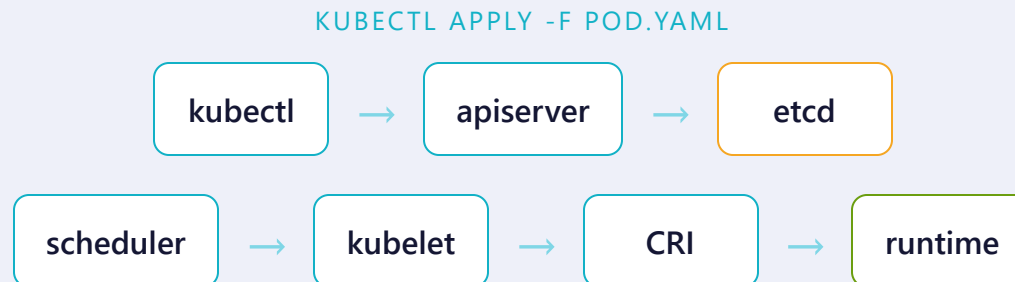
## MENTAL MODEL

The kubelet is the only worker-node component that talks to the control plane — kube-proxy and the runtime just take orders from it.

# The full topology



# Watch this chain for every create



Validate & persist to etcd → scheduler assigns a node → kubelet on that node calls the CRI → runtime starts the container → status flows back *through the apiserver*, not directly to the caller.

# Where people trip

- **apiserver is the ONLY etcd client.** It's the single choke point for every read and write in the cluster.
- **Components never talk to each other directly.** The scheduler doesn't call the kubelet — both just watch the apiserver.
- **Lose etcd, lose the cluster.** Objects, Secrets, config — there's no rebuilding it from the nodes. Back it up and test the restore.
- **On kubeadm, the kubelet starts control-plane Pods** from static manifests in /etc/kubernetes/manifests — not the scheduler.



## RECAP

# Cluster architecture in one breath

- Control plane = apiserver + etcd + scheduler + controller-manager; worker node = kubelet + kube-proxy + container runtime
- The apiserver is the only etcd client — every other component just watches it
- apiserver down → no changes, but running Pods keep serving; etcd down → cluster state is gone
- `kubectl apply` → apiserver → etcd → scheduler → kubelet → CRI → runtime

Next: **Container Runtime & the CRI**